Deep Reinforcement Learning

Temporal Difference Learning & Q-learning

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

SciML

Content

Content

- The TD learning concept
- Prediction using TD learning
- Policy improvement / control using TD learning
  - SARSA
  - Q-learning
- Double Q-learning
- n-step TD learning
  - n-step TD prediction
  - n-step TD control
  - n-step off-policy learning
  - TD(λ)

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1	Multi-armed bandits	Exploration-exploitation dilemma
2	Markov decision processes	Dynamics, rewards, policies
3	Dynamic programming	Optimal decision making with *full knowledge*
4	Monte Carlo methods	*Data-driven learning* from entire episodes
5	Temporal difference learning & Q-learning	*Data-driven learning* from individual transitions
	Deep-learning-based methods	
	Model-Based Control	
	Advanced Topics	

Lecture contents

The TD learning concept

Temporal-difference learning and the previous methods

The main aspects of our previous two categories of methods:

1. Monte Carlo (MC): **learning from experience**, possibly without knowledge of a model, e.g., *MC prediction* with exploring starts:

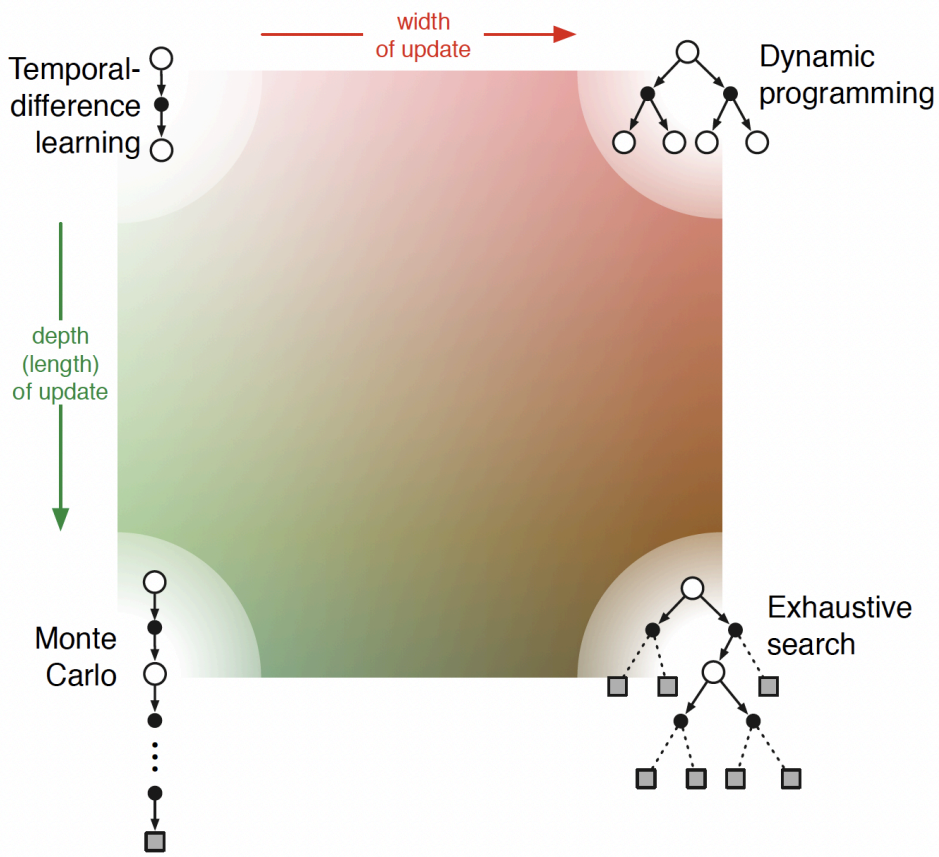
$$Q(s_t, a_t) = Q(s_t, a_t) + \frac{1}{n(s_t, a_t)} [g_t - Q(s_t, a_t)] .$$

2. Dynamic programming (DP): **bootstrapping** ⇒ updating estimates based on estimates, e.g., *iterative policy evaluation*:

$$V(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a) \left[r + \gamma V(s') \right] .$$

The **concept of temporal difference (TD) learning** is to combine both:

1. *Model-free* prediction and control in unknown MDPs.
2. Updates policy evaluation and improvement in an online fashion (i.e., not per episode) by *bootstrapping*.



(Sutton and Barto 1998, Figure 8.11)

💡 We are still considering finite MDPs here.

Prediction using TD learning

The general TD prediction update

Recall the **incremental update** of the type we discussed for multi-armed bandits or for MC control
$\Rightarrow$ **NewEstimate** $\leftarrow$ **OldEstimate** $+ \mathit{StepSize}$ [**target** $-$ **OldEstimate**]:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[g_t - V(s_t) \right]. \tag{1}$$

- $\alpha \in (0, 1)$ is the forget factor / step size.
- g_t is the target of the incremental update rule.
- To execute (1), we have to wait until the end of the episode to get g_t.

One-step TD / TD(0) update

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right]. \tag{2}$$

- Here, the **TD target** is $y_t = r_t + \gamma V(s_{t+1})$.
- TD is *bootstrapping*: estimate $V(s_t)$ based on $V(s_{t+1})$.
- *Delay time of one* time step; no need to wait until the end of the episode.

Algorithmic implementation: TD-based prediction

Algorithm: TD-based prediction.

Parameters: Step size $\alpha \in (0, 1)$

Initialize: $V(s)$ arbitrarily for $s \in \mathcal{S}$, $V(\textbf{terminal}) = 0$

for $k = 1, 2, \ldots, K$ episodes:

  for $t = 0, 1, \ldots, T$:

    Sample action $a_t \sim \pi(a|s)$ and apply

    Observe s_{t+1} and r_t

    $V(s_t) \leftarrow V(s_t) + \alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right]$ (Eq. (2))

    if s_{t+1} is terminal **then** STOP

💡 The algorithm can directly be applied to the prediction of action-value functions.

The TD error and its relation to the MC error

Using the target, we can define the **TD error*** $\delta \Rightarrow$ the expression inside the bracket of Eq. (2) we're using to improve our estimate:

$$\delta_t = \textbf{target} - V(s_t) = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Batch mode: Let's assume that the TD(0) estimate of $V(s)$ does not change within an episode, but that we apply all updates simulatenously once the episode is finished (exactly as we need to do in MC prediction):

$$\underbrace{g_t - V(s_t)}_{\text{MC error}} = r_t + \gamma g_{t+1} - V(s_t) + \gamma V(s_{t+1}) - \gamma V(s_{t+1})$$

$$= \delta_t + \gamma \underbrace{(g_{t+1} - V(s_{t+1}))}_{\text{MC error}} = \delta_t + \gamma \delta_{t+1} + \gamma^2 (g_{t+2} - V(s_{t+2}))$$

$$= \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} + \gamma^3 (g_{t+3} - V(s_{t+3})) = \ldots = \sum_{k=t}^{T} \gamma^{k-t} \delta_k.$$

- The MC error is the discounted sum of TD errors in this case.
- If $V(s)$ is updated during an episode (as is common in TD(0)), the above identity only holds approximately.

* For sufficiently small step size α, it can be shown that the TD error conveges to zero in batch mode (Sutton and Barto 1998).

Convergence of TD(0)

Given a finite MDP and a fixed policy π, the state-value estimate V of TD(0) converges to the true V^π
(that is, $\lim_{t\to\infty} \delta_t = 0$)

- in the mean for a constant but sufficiently small step-size α and
- with probability one if the step-size satisfies the *Robbins-Munro* condition

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \text{and} \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty. \tag{3}$$

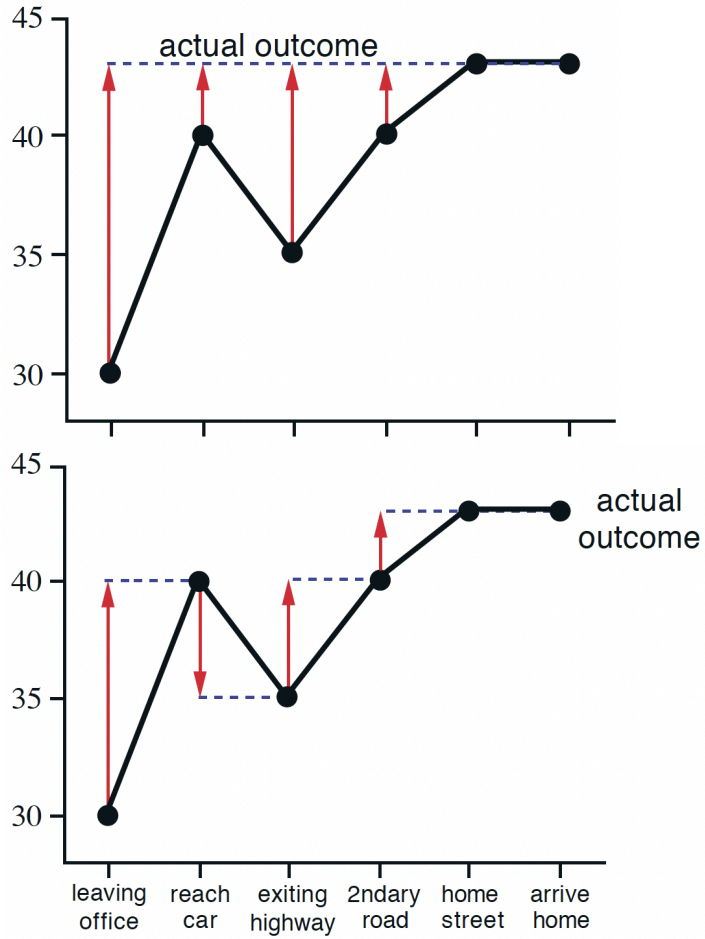
- In particular, $\alpha_t = \frac{1}{t}$ meets the condition (3).
- TD(0) often converges faster than MC, as we will see in the following examples. However, there is no guarantee it's faster.
- TD(0) can be more sensitive to poor initializations $V_0(s)$ compared to MC.

The *driving home* example

Imagine you want to predict your time to get home after work, starting with an estimate of 30 minutes.

State	Elapsed Time (minutes)	Predicted Time to Go	Predicted Total Time
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

Overview of elapsed time and your predictions of time to go and total time (Sutton and Barto 1998, Example 6.1)



MC vs. TD estimate.

- In MC, we estimate the returns g_t **after** we have reached home ⇒ it is hard to assess where the deviation was caused.
- In TD, we can immediately shift our estimate in each time step.
- With TD, we can even learn from continuing tasks!

The batch training AB example (1)

Assume that we have an MDP with only two states: A and B. We do not consider discounting (i.e., $\gamma = 1$).

We collect eight samples from the dynamics, and we would like to estimate the values $V(A)$ and $V(B)$:

Episode k	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Sequence (s, r, s, r)	A, 0, B, 0	B, 1	B, 1	B, 1	B, 1	B, 1	B, 1	B, 0

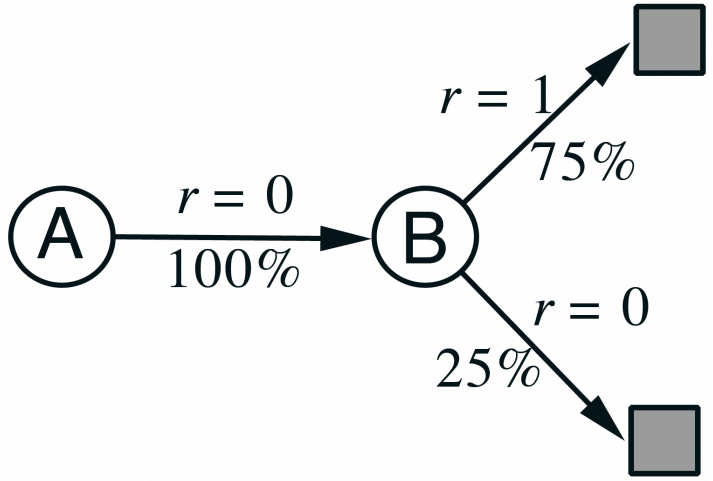
Sample of $K = 8$ trajectories, including the rewards following the states.

Question: What's $V(A)$ and $V(B)$ based on

- the MC estimate?
- the TD(0) estimate?

Before approaching this, let's try to model the underlying MDP!

- B occurs eight times, follwed by $r = +1$ in six, and by $r = 0$ in two cases. $\Rightarrow V(B) = \frac{3}{4}$.
- What about $V(A)$?
  1. In our example, the only episode containing A has return $g = 0$ $= 0$. $\Rightarrow V(A)$
  2. On the other hand, A is followed by $r = 0$ and then $s = B$ $= V(B) = \frac{3}{4}$. $\Rightarrow V(A)$



Option 2. for modeling the MDP (Sutton and Barto 1998, Ex. 6.4).

Answer 1. is the MC estimate, answer 2. is the TD estimate!

The batch training AB example (2)

Let's formally calculate the MC and TD updates:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[g_t - V(s_t) \right] \quad \text{(MC)}$$
$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right] \quad \text{(TD)}$$

Convergence: no change in the estimate, i.e.,

$$\alpha \left[g_t - V(s_t) \right] = g_t - V(s_t) = 0 \quad \text{(MC)}$$
$$\alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right] = r_t + \gamma V(s_{t+1}) - V(s_t) = 0 \quad \text{(TD)}$$

Consider a batch sweep over the the K episodes:

$$\sum_{k=1}^{K} g_{t,k} - V(s_{t,k}) = 0 \quad \text{(MC)}$$

$$\sum_{k=1}^{K} r_{t,k} + \gamma V(s_{t+1,k}) - V(s_{t,k}) = 0 \quad \text{(TD)}$$

The batch training AB example (3)

Apply the previous equations first to **state B**. Since B is a terminal state, $V(s_{t+1}) = 0$ and $g_{t,k} = r_{t,k}$ apply, i.e., the MC and TD updates are identical for B:

$$\text{MC}\big|_{s=B}: \quad \sum_{k=1}^{K} g_{t,k} - V(B) = 0 \quad \Leftrightarrow \quad V(B) = \frac{1}{K}\sum_{k=1}^{K} r_{t,k} = \frac{6}{8} = \frac{3}{4},$$

$$\text{TD}\big|_{s=B}: \quad \sum_{k=1}^{K} r_{t,k} - V(B) = 0 \quad \Leftrightarrow \quad V(B) = \frac{1}{K}\sum_{k=1}^{K} r_{t,k} = \frac{6}{8} = \frac{3}{4}.$$

Now consider **state A**, where we have only one trajectory:

- the instantaneous reward following A is zero, and the return of that episode is also zero.
- The TD bootstrap estimate of B is $V(B) = \frac{3}{4}$.

$$\text{MC}\big|_{s=A}: \quad \sum_{k=1}^{K} g_{t,k} - V(A) = 0 \quad \Leftrightarrow \quad V(A) = 0,$$

$$\text{TD}\big|_{s=A}: \quad \sum_{k=1}^{K} r_{t,k} + \gamma V(B) - V(A) = \sum_{k=1}^{K} \gamma V(B) - V(A) = 0 \quad \Leftrightarrow \quad V(A) = \gamma V(B) = \frac{3}{4}.$$

Certainty equivalence

Where does this difference come from? $\Rightarrow$ Without going into a detailed derivation:

- MC batch learning converges to the least **squares fit of the sampled returns**:

$$\sum_{k=1}^{K}\sum_{t=1}^{T}(g_{t,k} - V(s_{t,k}))^2.$$

- TD batch learning converges to the **maximum likelihood estimate** such that $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ explains the data with highest probability:

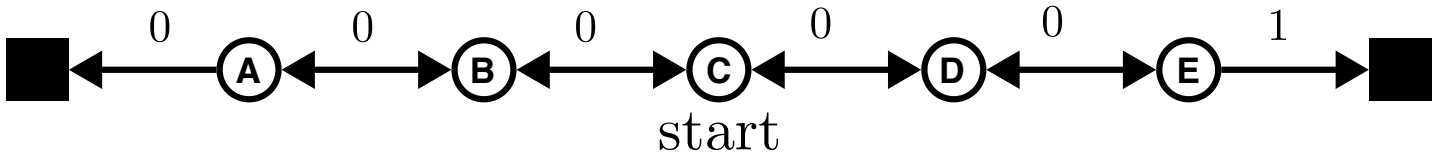
$$\hat{p}(s'|s,a) = \frac{1}{n(s,a)}\sum_{k=1}^{K}\sum_{t=1}^{T}\mathbb{1}(s_{t+1,k} = s'|s_{t,k} = s, a_{t,k} = a),$$

$$\hat{r} = \frac{1}{n(s,a)}\sum_{k=1}^{K}\sum_{t=1}^{T}\mathbb{1}(s_{t,k} = s, a_{t,k} = a)r_{t,k}.$$

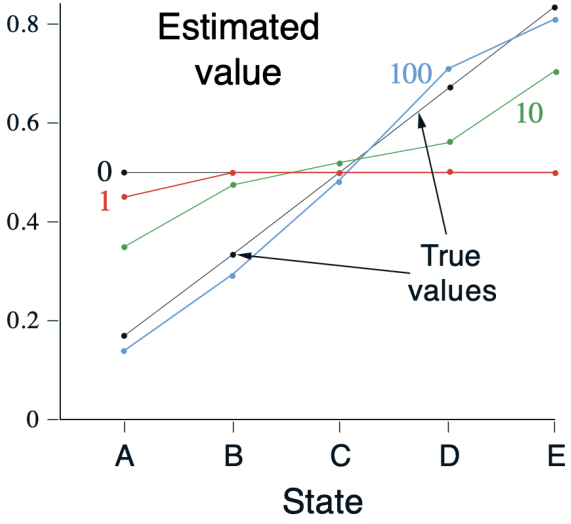
$\Rightarrow$ *TD assumes an MDP problem structure* and is absolutely certain that its internal model concept describes the real world perfectly (so-called **certainty equivalence**).

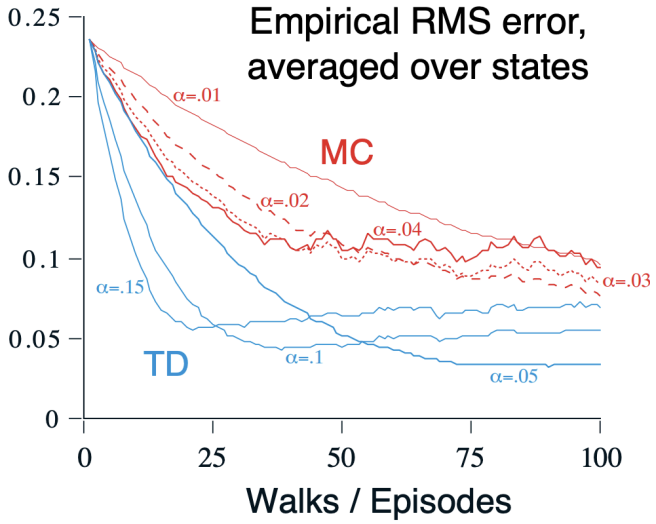
The *random walk* example

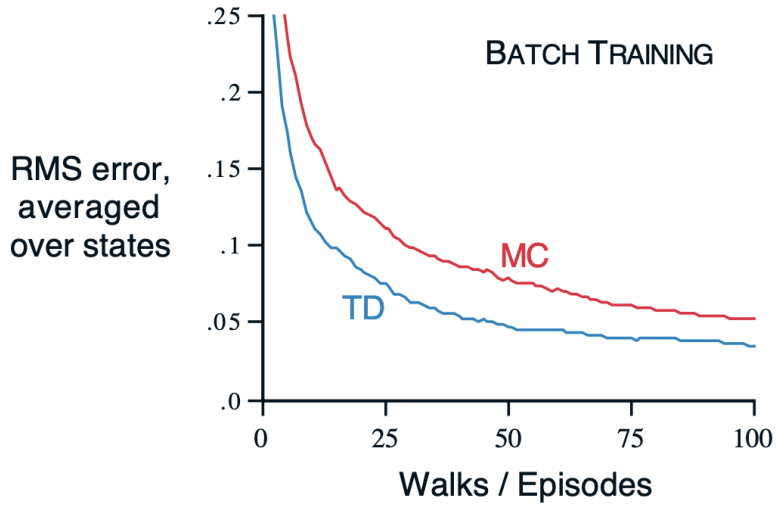
Consider a simple Markov reward process (MRP; no actions), for which we want to estimate $V(s)$ from experience only.



- For all states s, we have $p(\leftarrow) = p(\rightarrow) = 0.5$.
- The MRP is undiscounted ($\gamma = 1$).
- The value of each state is the probability of terminating on the right: $V(A/B/C/D/E) = \frac{1}{6}$ / $\frac{1}{3}$ / $\frac{1}{2}$ / $\frac{2}{3}$ / $\frac{5}{6}$.





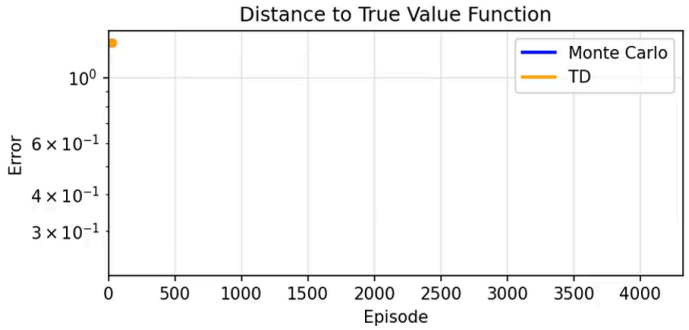


*TD(0) learning for the random walk. **Left**: Different numbers of iterations (α=0.1). **Middle**: Learning curves for different values of α. **Right**: Batch training.*

The example is taken from (Sutton and Barto 1998, Ex. 6.2)

Example: MC vs. TD(0) prediction in Gridworld





- Fixed policy:

$$\pi(\cdot|s) = [0.25, 0.25, 0.25, 0.25]^\top \; \forall \, s \in \mathcal{S}.$$

- Discount: $\gamma = 0.9$.

Why is MC learning faster? Wouldn't we expect TD(0) to be stronger?

$\Rightarrow$ We update entire trajectories, not just the last field we saw before the target.

$\Rightarrow$ Also, we started with a poor initializaiton for V.

Policy improvement / control using TD learning

Application of generalized policy iteration to TD learning

The GPI concept can directly be applied to the TD framework by replacing values with action-values, where we alternate between estimation (i.e., prediction) and improvement:

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \dots \xrightarrow{I} \pi^* \xrightarrow{E} Q^* = Q^{\pi^*}.$$

Estimation: TD(0) update of the Q-function

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \underbrace{\left[\overbrace{r_t + \gamma Q(s_{t+1}, a_{t+1})}^{\text{TD target } y} - Q(s_t, a_t)\right]}_{\text{TD error } \delta}. \tag{4}$$

The policy: ϵ-greedy / ϵ-soft*.

Convergence (Sutton and Barto 1998)

- towards Q^* with probability one if all tuples (s, a) are visited infinitely often and the step-size condition (3) is satisfied.
- towards π^* if the policy is GLIE (**G**reedy in the **L**imit with **I**nfinite **E**xploration).

Key question: how do we select a_{t+1}? $\Rightarrow$ **On-policy versus off policy!**

* $\pi(a|s) > 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$.

On-policy TD control: SARSA

s, a, r, s', a': State-Action-Reward-Next State-Next Action

Algorithm: SARSA.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\textbf{terminal}, \cdot) = 0$
- $\pi = \epsilon$-greedy(Q)

for $k = 1, 2, \ldots, K$ episodes:
Initialize $s_t \leftarrow s_0, t \leftarrow 0$
while s_t is not terminal:
Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})
Select $a_{t+1} \sim \pi(s_{t+1})$
Update Q given $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

Update policy: $\pi = \epsilon$-greedy(Q) (This is on-policy!)
$t \leftarrow t + 1$

Convergence: SARSA for finite-state and finite-action MDPs converges to the optimal action-value,

$$Q(s, a) \rightarrow Q^*(s, a),$$

under the following conditions:

1. The policy sequence $\pi_t(a|s)$ is GLIE.
2. The step-sizes α_t satisfy the step-size condition (3) (as does, e.g., $\alpha_t = \frac{1}{t}$).

Off-policy TD control: Q-learning

Algorithm: Q-learning.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\textbf{terminal}, \cdot) = 0$
- $\pi = \epsilon$-greedy(Q)

for $k = 1, 2, \ldots, K$ episodes:
Initialize $s_t \leftarrow s_0, t \leftarrow 0$
while s_t is not terminal:
Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})
~~Select $a_{t+1} \sim \pi(s_{t+1})$~~ (Not needed in off-policy)
Update Q given (s_t, a_t, r_t, s_{t+1}):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon$-greedy(Q)
$t \leftarrow t + 1$

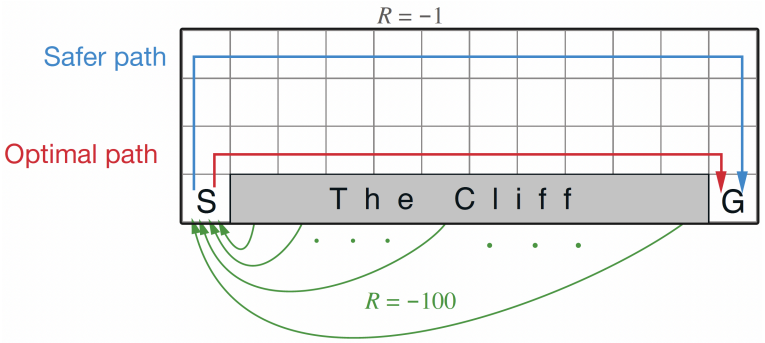
- Q-learning is similar but directly estimates Q^*.
- *Off-policy update*, since the optimal action-value function is updated independent of a given behavior policy.
- The policy still determines which state–action pairs are visited and updated.
- Convergence with probability one to Q^* (Sutton and Barto 1998) if
  - all state-action pairs continue to be visited,
  - a variant of the usual stochastic approximation conditions on the sequence of step-size parameters holds.

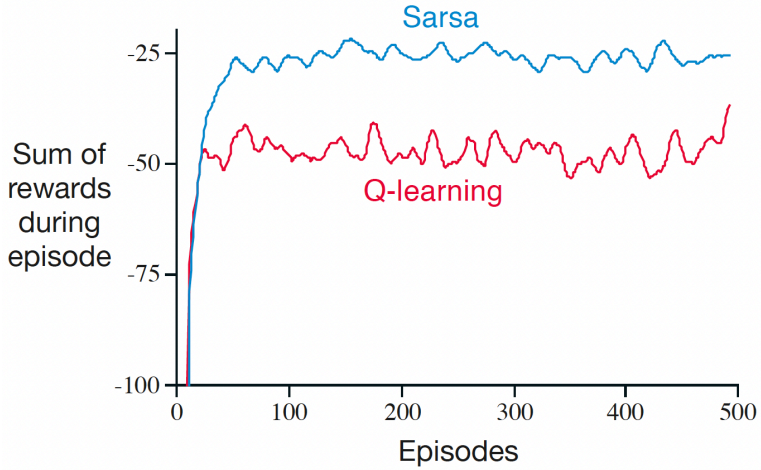
Example: Walking along a cliff

- Usual Gridworld setting.
- All rewards are -1, except
  - the goal G has $r = 0$,
  - the cliff has $r = -100$ and leads back to S.
- The policy is ϵ-greedy with $\epsilon = 0.1$

Outcome:

- Q-learning learns the optimal path, close along the cliff.
- On-policy SARSA learns a much safer path. *Why do you think that is*?
- In the GLIE case (i.e., $\epsilon \to 0$), both policies will converge to the optimal path.





In our setting, Q-learning is worse than SARSA. *Why do you think that is*?

$\Rightarrow$ the combination of falling off randomly and the resulting very large negative reward.

The example is taken from (Sutton and Barto 1998, Ex. 6.6)

Example: Gridworld



On-policy: SARSA



Off-policy: Q-learning

- Discount: $\gamma = 0.9$
- Exploration: $\epsilon = 0.2$
- Step size: $\alpha = 0.5$

Expected SARSA

What can we do to reduce the large variance in the SARSA update

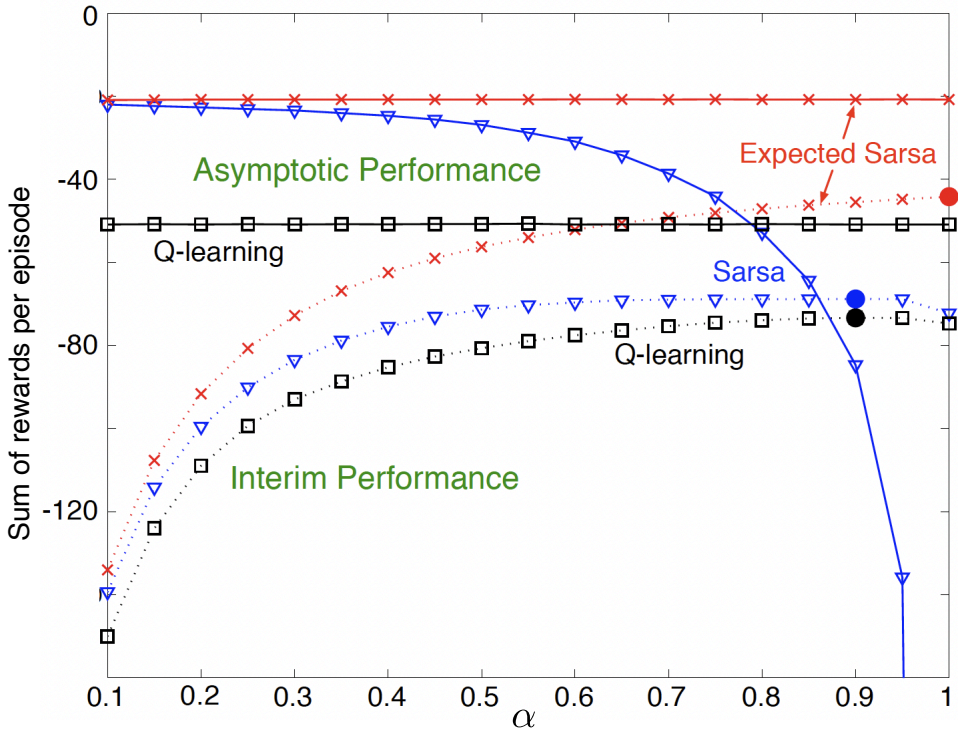
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]?$$

Let's take the expectation of Q $\rightarrow$ **expected SARSA**:

$$\begin{aligned} Q(s_t, a_t) &\leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \mathbb{E}_\pi \left[Q(s_{t+1}, a) | s_t \right] - Q(s_t, a_t) \right] \\ &= Q(s_t, a_t) + \alpha \left[r_t + \gamma \left(\sum_{a \in \mathcal{A}} \pi(a | s_{t+1}) Q(s_{t+1}, a) \right) - Q(s_t, a_t) \right]. \end{aligned}$$

For comparison: Q-learning

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{\mathbf{a} \in \mathcal{A}} \mathbf{Q(s_{t+1}, a)} - Q(s_t, a_t) \right]$$



Double Q-learning

Maximization bias

All control algorithms discussed so far involve *maximization operations*:

- Q-learning: target policy is greedy and directly uses max operator for action-value updates.
- SARSA: typically uses an ϵ-greedy framework $\rightarrow$ max updates during policy improvement.

This can lead to a significant *positive bias*:

- Maximization over sampled values is used implicitly as an estimate of the maximum value.
- This issue is called **maximization bias**.

Small example:

- Consider a single state s with multiple possible actions a.
- The true action values are all $Q(s, a) = 0$.
- The sampled estimates $\hat{Q}(s, a)$ are uncertain, i.e., randomly distributed. Some samples are above and below zero.
- Consequence: The maximum of the estimate is positive!

Double Q-learning approach

Split the learning process:

- Divide sampled experience into two sets.
- Use sets to estimate independent estimates $Q_1(s, a)$ and $Q_2(s, a)$.

Assign specific tasks to each estimate:

- Estimate the maximizing action using Q_1:

$$a^* = \arg\max_{a \in \mathcal{A}} Q_1(s, a).$$

- Estimate corresponding action value using Q_2:

$$Q(s, a^*) \approx Q_2(s, a^*) = Q_2(s, \arg\max_{a \in \mathcal{A}} Q_1(s, a)).$$

- Since this approach is perfectly symmetric, we can repeat the process with reversed roles between Q_1 and Q_2.

Double Q-learning algorithm

Algorithm: Q-learning.
initialize

- $Q_1(s, a)$ and $Q_2(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q_1(\text{terminal}, \cdot) = Q_2(\text{terminal}, \cdot) = 0$

for $k = 1, 2, \ldots, K$ episodes:
Initialize $s_t \leftarrow s_0, t \leftarrow 0$
while s_t is not terminal:
Take action a_t using an ϵ-greedy policy on $Q_1 + Q_2$ and observe (r_t, s_{t+1})
if $\text{rand}() > 0.5$ **then**

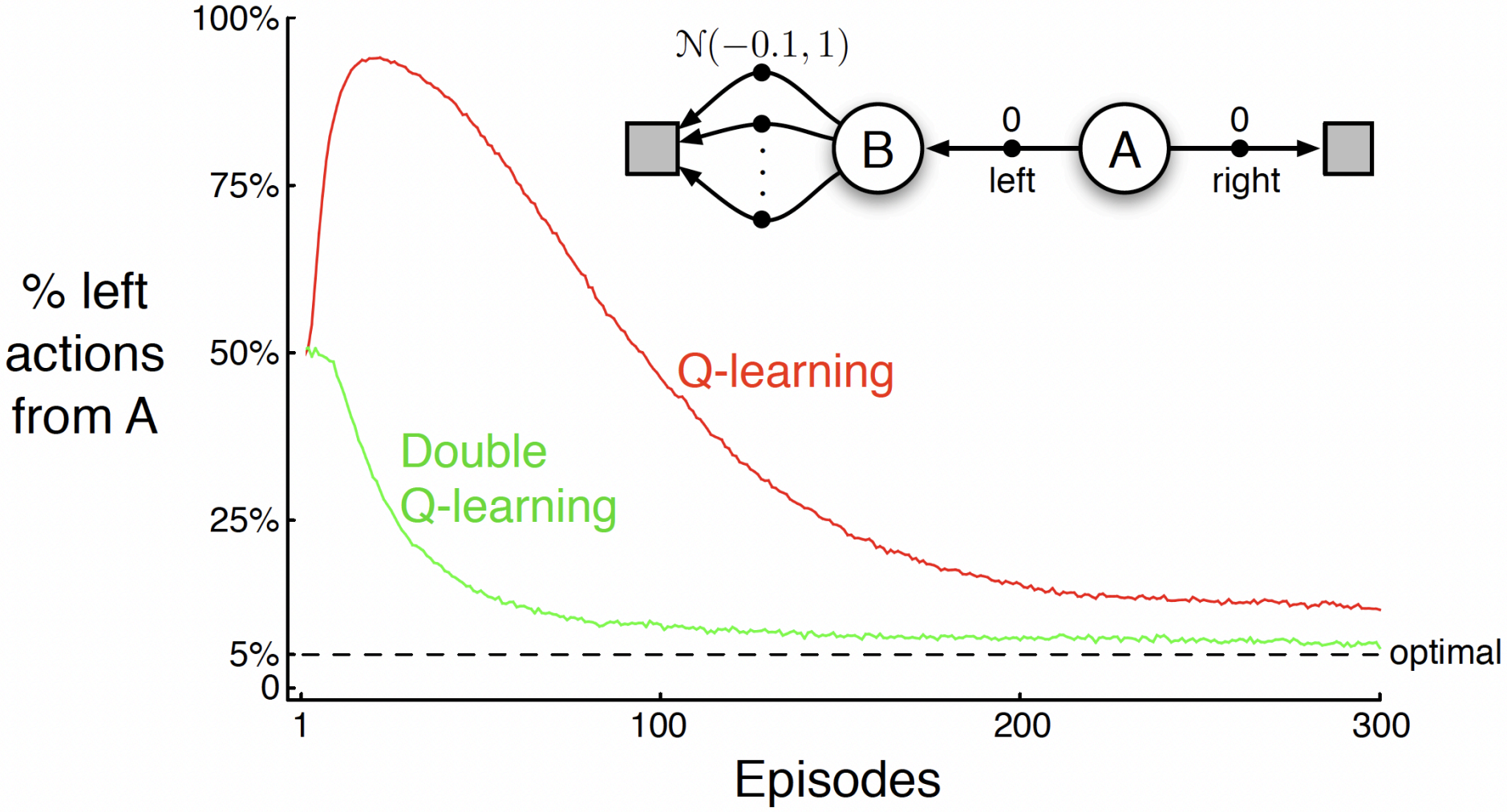
$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha \left[r_t + \gamma Q_2(s_{t+1}, \arg\max_a Q_1(s_{t+1}, a)) - Q_1(s_t, a_t) \right]$$

else:

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha \left[r_t + \gamma Q_1(s_{t+1}, \arg\max_a Q_2(s_{t+1}, a)) - Q_2(s_t, a_t) \right]$$

$t \leftarrow t + 1$

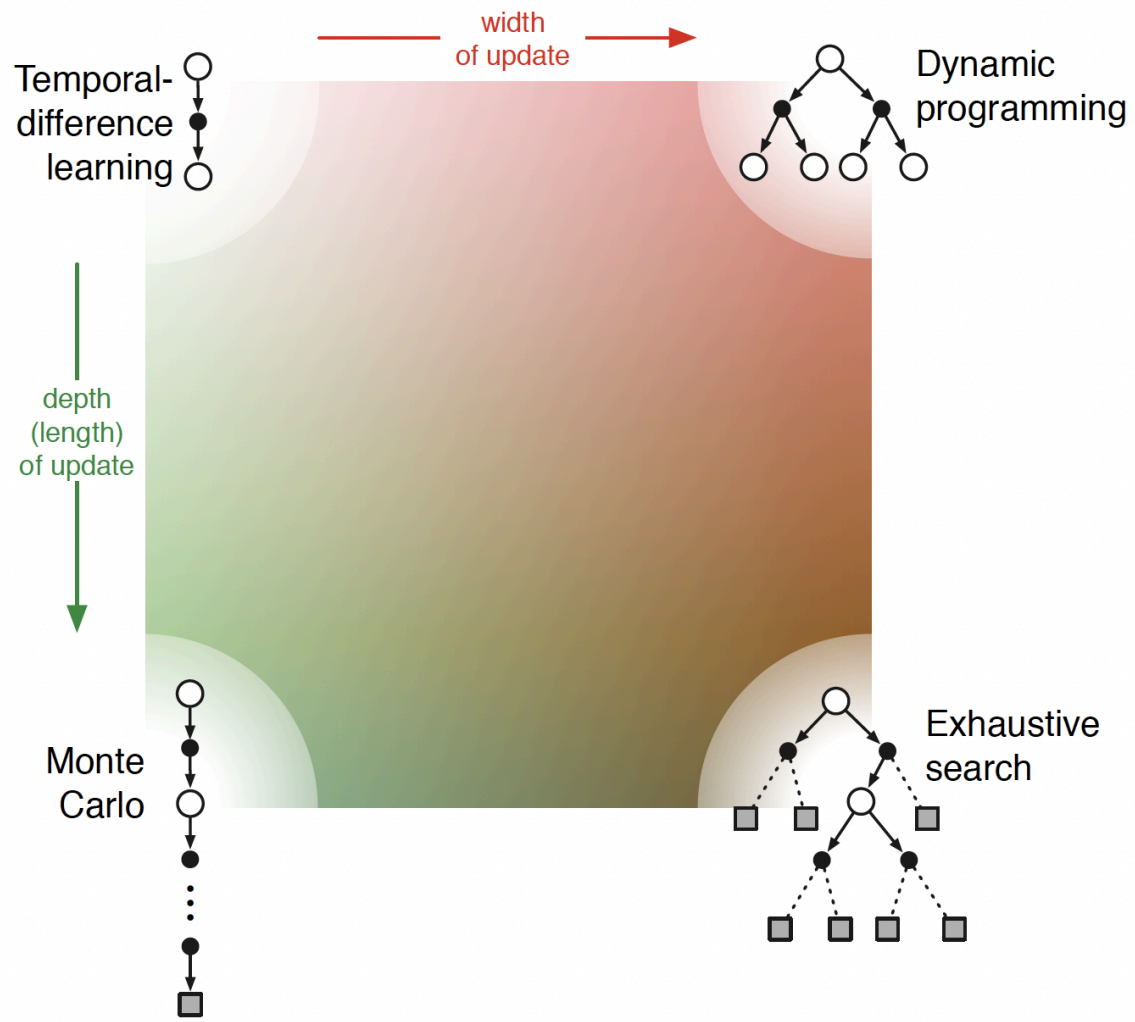
Maximization bias example



Comparison of Q-learning and Double Q-learning on a simple episodic MDP (Sutton and Barto 1998, Ex. 6.7). Q-learning initially learns to take the left action much more often than the right action, and always takes it significantly more often than the 5% minimum probability enforced by ϵ-greedy action selection with $\epsilon = 0.1$. In contrast, Double Q-learning is essentially unaffected by maximization bias. These data are averaged over 10,000 runs. The initial action-value estimates were zero.

n-step TD learning

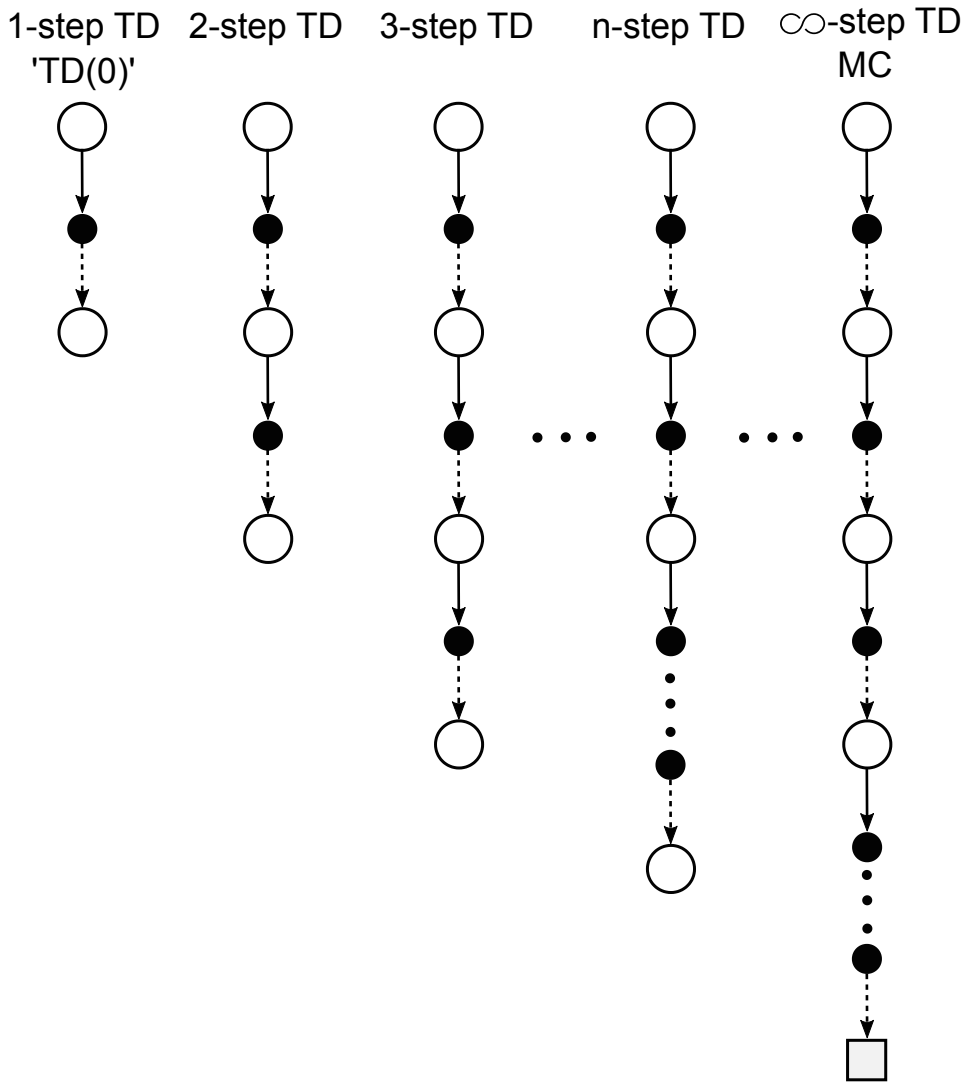
Let's unify MC and TD learning



(*Sutton and Barto 1998*, *Figure 8.11*)

- MC and TD are the "extreme options" in terms of the update depth.
- What about intermediate solutions?

n-step bootstrapping idea



(Sutton and Barto 1998, Figure 7.1)

- n-step update: consider n rewards plus estimated value n steps later (bootstrapping).
- Consequence: Estimate update is available only after an n-step delay.
- TD(0) and MC are special cases included in n-step prediction.

Unified target notation (1)

Recap the **update targets** for the incremental prediction methods:

- **Monte Carlo**: builds on the complete sampled return series

$$g_{t:T} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots + \gamma^T r_T.$$

  - $g_{t:T}$ denotes that all steps until termination at T are considered to estimate a target for step t.

- **TD(0)**: utilizes a one-step bootstrapped return

$$g_{t:t+1} = r_t + \gamma V_t(s_{t+1}).$$

  - $g_{t:t+1}$ highlights that only one future sampled reward step is considered before bootstrapping.
  - V_t is an estimate of V^π at time step t.

Unified target notation (2)

Definition: n-step state-value prediction target

The generalized n-step target is defined as:

$$g_{t:t+n} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma^n V_{t+n-1}(s_{t+n}).$$

- Approximation of full return series truncated after n-steps.
- If $t + n \geq T$ (i.e., n-step prediction exceeds termination lookahead), then all missing terms are considered zero.

Definition: n-step TD

The state-value estimate using the n-step return approximation is:

$$V_{t+n}(s_t) = V_{t+n-1}(s_t) + \alpha \left[g_{t:t+n} - V_{t+n-1}(s_t) \right], \qquad 0 \leq t \leq T.$$

- Delay of n-steps before $V(s)$ is updated.
- Additional auxiliary update steps required at the end of each episode.

n-step TD prediction: Algorithmic implementation

Algorithm: n-step TD for estimating $V \approx V^\pi$.

input. a policy π to be evaluated, parameter: step size $\alpha \in (0, 1]$, prediction steps $n \in \mathbb{Z}^+$.

Initialize: $V(s)$ arbitrarily for $s \in \mathcal{S}$, $V(\textbf{terminal}) = 0$.

for $k = 1, 2, \ldots, K$ episodes:

initialize and store s_0

$T \leftarrow \infty, \tau \leftarrow 0$

for $t = 1, 2, 3, \ldots$

if $t < T$ **then**

take action $a_t \sim \pi(\cdot|s_t)$, observe and store s_{t+1} and r_{t+1}

if s_{t+1} is **terminal** **then** $T \leftarrow k + 1$

$\tau \leftarrow t - n - 1$ (τ is the time index for the estimate update)

if $\tau \geq 0$ **then**

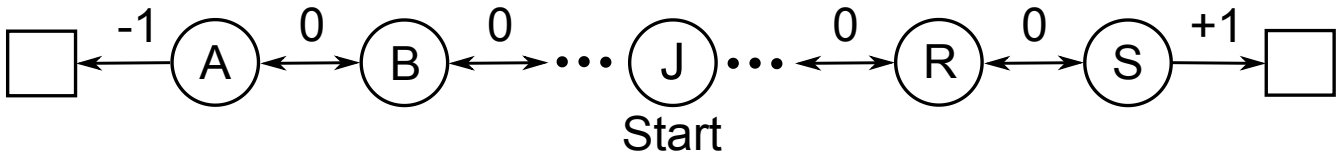
$g \leftarrow \sum_{k=\tau+1}^{\min(\tau+n, T)} \gamma^{k-\tau-1} r_k$

if $\tau + n < T$ **then** $g \leftarrow g + \gamma^n V(s_{\tau+n})$

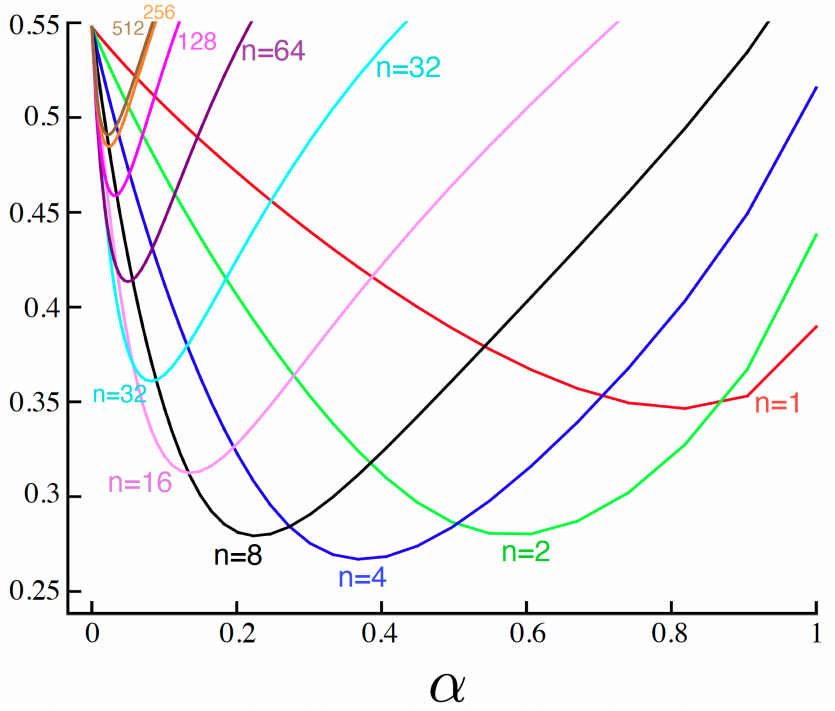
$V(s_\tau) \leftarrow V(s_\tau) + \alpha [g - V(s_\tau)]$

if $\tau = T - 1$ **then** STOP

Example: 19-state random walk



Average RMS error over 19 states and first 10 episodes



(Sutton and Barto 1998, Example 7.1)

- Early stage performance after only 10 episodes
- Averaged over 100 independent runs
- Best result here: $n = 4, \alpha \approx 0.4$
- Picture may change for longer episodes (no generalizable results)

n-step TD control

Transfer the n-step approach to state-action values (1)

- For on-policy control by SARSA action-value estimates are required.
- Recap the one-step action-value update as required for "SARSA(0)":

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[\underbrace{r_t + \gamma Q(s_{t+1}, a_{t+1})}_{\text{target } g} - Q(s_t, a_t) \right]$$

Definition: n-step state-action value prediction target.
Analog to n-step TD, the state-action value target (with $a_{t+n} \sim \pi(\cdot|s_{t+n})$) is rewritten as:

$$g_{t:t+n} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma^n Q_{t+n-1}(s_{t+n}, a_{t+n}). \tag{5}$$

Definition: n-step state-action value prediction target.
For n-step expected SARSA, the update is similar but we're considering the expected value at $t+n$:

$$g_{t:t+n} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma^n \sum_{a \in \mathcal{A}} \pi(a|s_{t+1}) Q_{t+n-1}(s_{t+n}, a). \tag{6}$$

💡 Again, if an episode terminates within the lookahead horizon ($t + n \geq T$) the target is equal to the Monte Carlo update: $g_{t:t+n} = g_t$.

Transfer the n-step approach to state-action values (2)

Finally, the modified n-step targets can be directly integrated to the state-action value estimate update rule of SARSA:

Definition: n-step SARSA.

$$Q_{t+n}(s_t, a_t) = Q_{t+n-1}(s_t, a_t) + \alpha \left[g_{t:t+n} - Q_{t+n-1}(s_t, a_t) \right], \qquad 0 \leq t \leq T,$$

with $g_{t:t+n}$ according to (5) for the *on-policy*, and (6) for the *expected* version of SARSA.

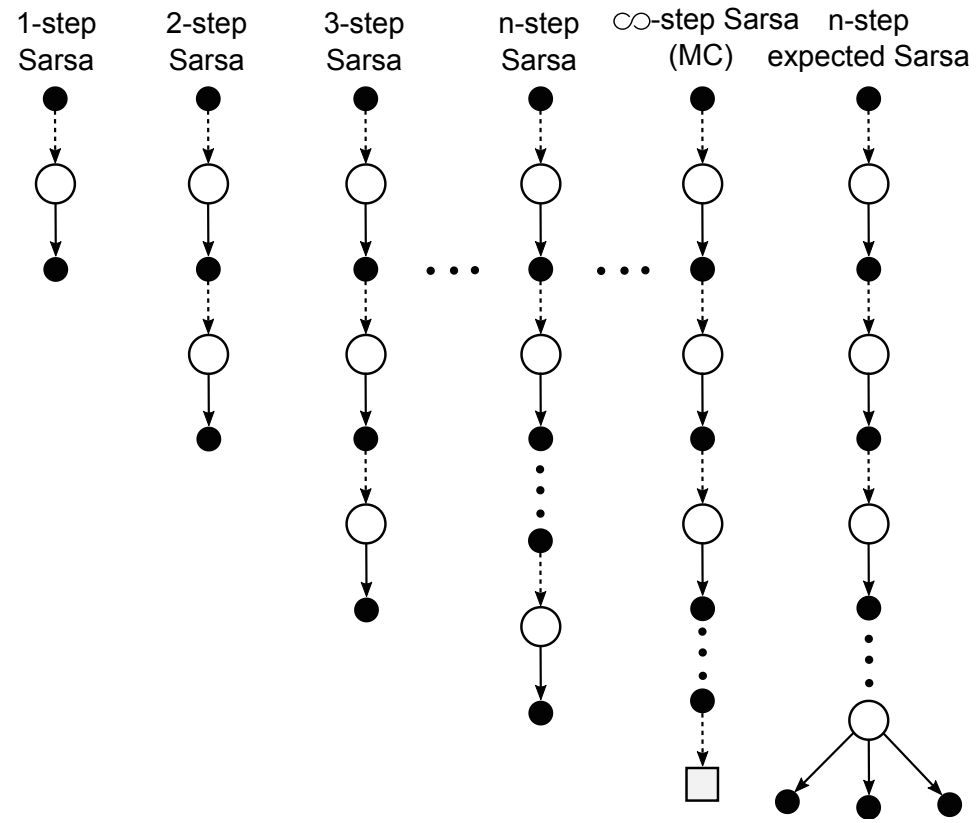
Remark: Off-policy n-step SARSA.

For the off-policy version of n-step SARSA, we need to adapt the importance sampling weights we have seen in MC off-policy control to the n-step case, i.e., for various prediction lengths $h \in \{t+1, \ldots, t+n-1\}$:

$$\rho_{t:h} = \prod_{k=t}^{\min(t+h, T)} \frac{\pi\left(a_k | s_k\right)}{b(a_k | s_k)}.$$

For more details, see (Sutton and Barto 1998, Chapter 7.3).

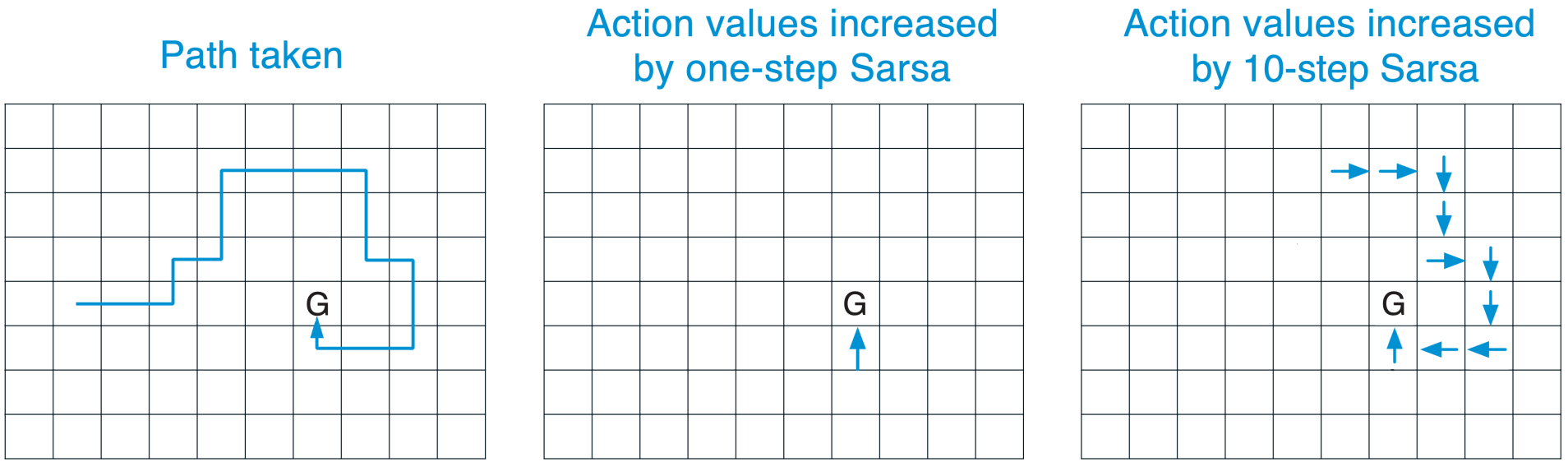
n-step bootstrapping for state-action values



(Sutton and Barto 1998, Figure 7.3)

Example: Gridworld

- Standard assumptions for movement etc.
- zero rewards everywhere, except for the goal G, where we have $r = 1$.

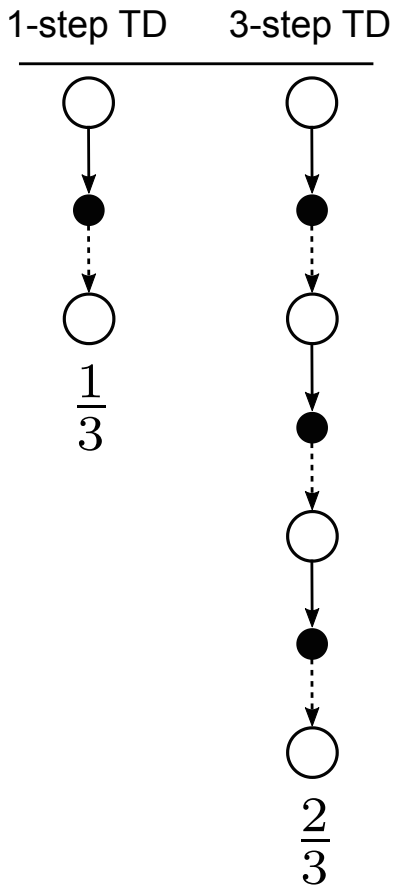


Executed updates (highlighted by arrows) for one-step and ten-step SARSA implementations during an episode. (Sutton and Barto 1998, Figure 7.4)

- One-step SARSA: one Q-value is updated $\Rightarrow$ This is the only field where we can "see" the reward of reaching G.
- Ten-step SARSA> ten Q-values are updated.
- Consequence: a trade-off between the resulting learning delay and the number of updated state-action values results.

TD(λ)

Averaging of n-step returns



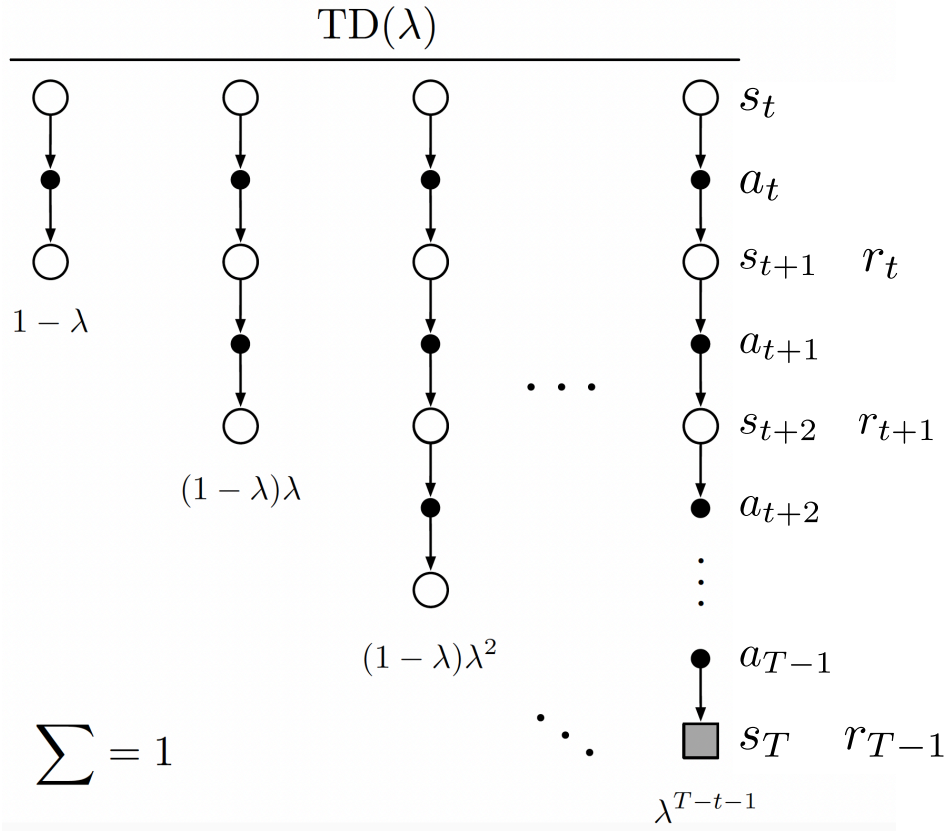
Exemplary averaging of one- and three-step returns. (Sutton and Barto 1998, Figure 7.4)

- Averaging different n-step returns is possible without introducing a bias (if sum of weights is one).
- Example on the left:

$$g = \frac{1}{3} g_{t:t+1} + \frac{2}{3} g_{t:t+3}.$$

- Horizontal line in backup diagram indicates the averaging.
- Enables additional degree of freedom to reduce prediction error.
- Such updates are called **compound updates**.

λ-return (1)



The backup diagram for TD(λ). If λ = 0, then the overall update reduces to its first component, the one-step TD(0) update, whereas if λ = 1, then the overall update reduces to its last component, the Monte Carlo update. (Sutton and Barto 1998, Figure 12.1)

- **λ-return**: is a compound update with exponentially decaying weights:

$$g_t^\lambda = (1-\lambda)\sum_{k=1}^{\infty} \lambda^{(k-1)} g_{t:t+n}. \quad (7)$$

- Parameter is $\lambda \in [0, 1]$.
- Geometric series of weights is one:

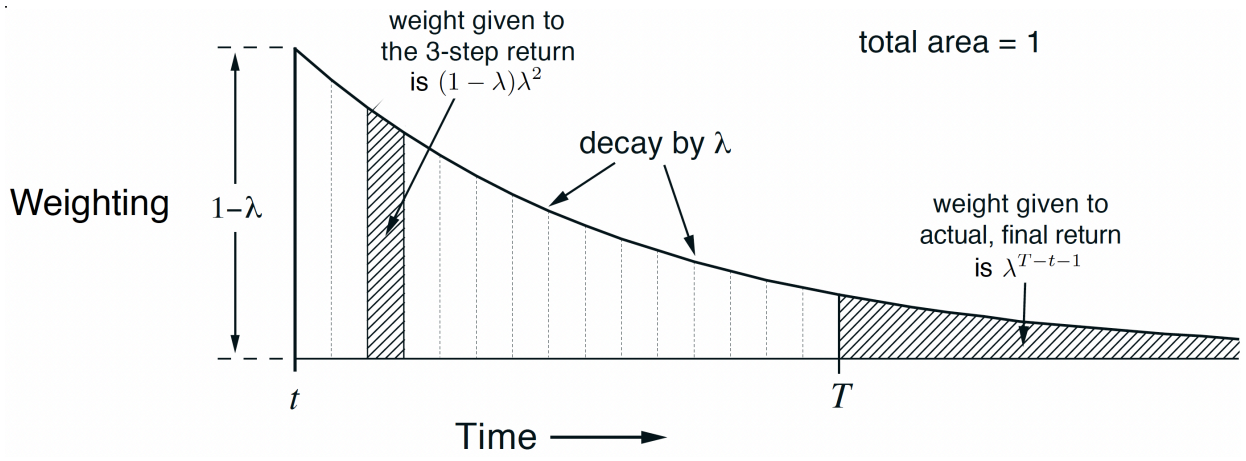
$$(1-\lambda)\sum_{k=1}^{\infty} \lambda^{(k-1)} = 1.$$

λ-return (2)

- Rewrite λ-return for episodic tasks with termination at $t = T$:

$$g_t^\lambda = (1-\lambda)\left(\sum_{k=1}^{T-t-1} \lambda^{(k-1)} g_{t:t+n}\right) + \lambda^{T-t-1} g_t. \tag{8}$$

- Return g_t after termination is weighted with residual weight λ^{T-t-1}.
- The equation includes two special cases:
  - If $\lambda = 0$: becomes the TD(0) update.
  - If $\lambda = 1$: becomes the MC update.



Weighting given in the λ-return to each of the n-step returns. (Sutton and Barto 1998, Figure 12.2)

Truncated λ-returns for continuing tasks

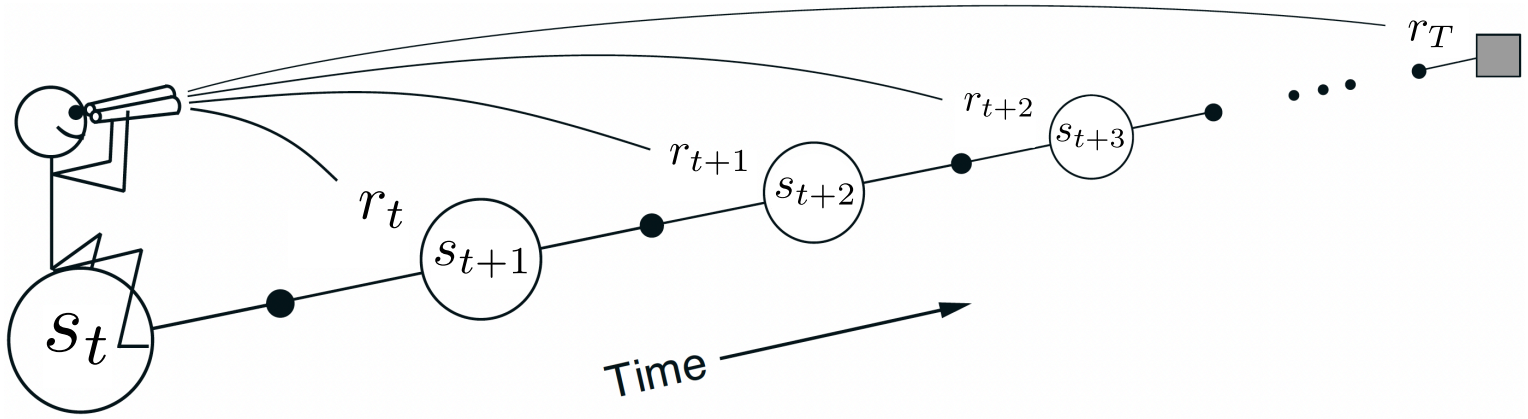
- Using λ-returns as in (7) is not feasible for continuing tasks.
- One would have to wait infinitely long to receive the trajectory.
- Intuitive approximation: truncate λ-return after h steps.
- This gives us a formulation very close to the episodic formulation (8):

$$g_{t:h}^{\lambda} = (1-\lambda)\left(\sum_{k=1}^{h-t-1} \lambda^{(k-1)} g_{t:t+n}\right) + \lambda^{h-t-1} g_{t:h}.$$

- Horizon h divides continuing tasks in rolling episodes.

Forward view of TD(λ)

- Both, n-step and λ-return updates, are based on a *forward view*.
- We have to wait for future states and rewards to arrive before we are able to perform an update.
- Currently, λ-returns are only an alternative to n-step updates with different weighting options.



The forward view. We decide how to update each state by looking forward to future rewards and states. (Sutton and Barto 1998, Figure 12.4).

Backward view of TD(λ)

General idea:

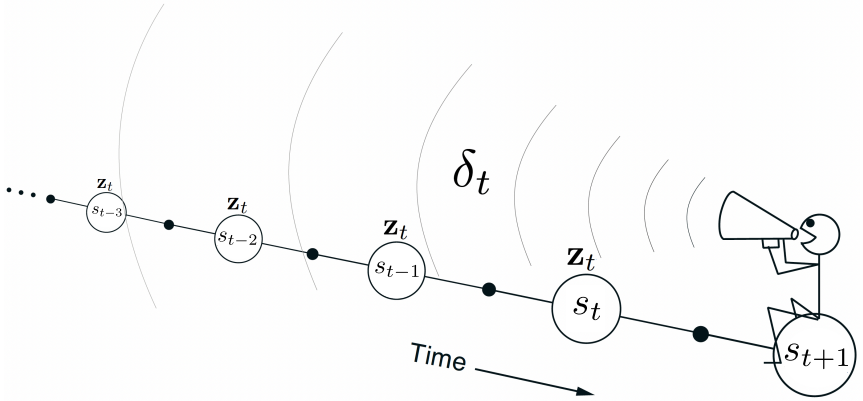
- Use λ-weighted returns looking into the past.
- Implement this in a recursive fashion to save memory
  $\Rightarrow$ Introduce an **eligibility trace** z_t denoting the importance of past events to the current state update:

$$z_0(s) = 0 \qquad \text{and} \qquad z_t(s) = \gamma\lambda z_{t-1}(s) + \begin{cases} 0 & \text{if } s_t \neq s \\ 1 & \text{if } s_t = s \end{cases}.$$

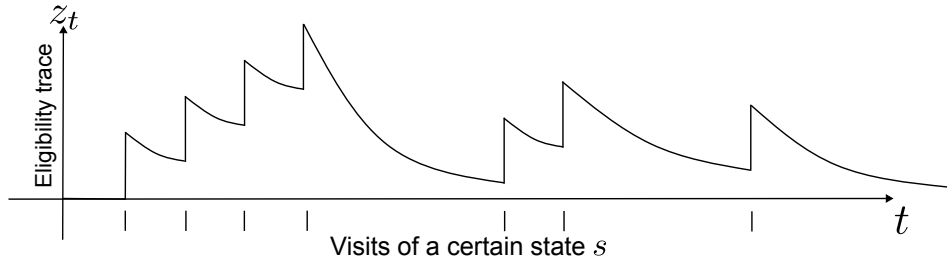
- We additionally scale the update term by $z(s)$, i.e., $\alpha \cdot [\textbf{target}(s) - \textbf{OldEstimate}(s)] \cdot z(s)$. For example,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \cdot [r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \cdot z(s_t, a_t).$$

- Instead of updating only one state-action pair (s_t, a_t) we update along the entire *trace* we have left.



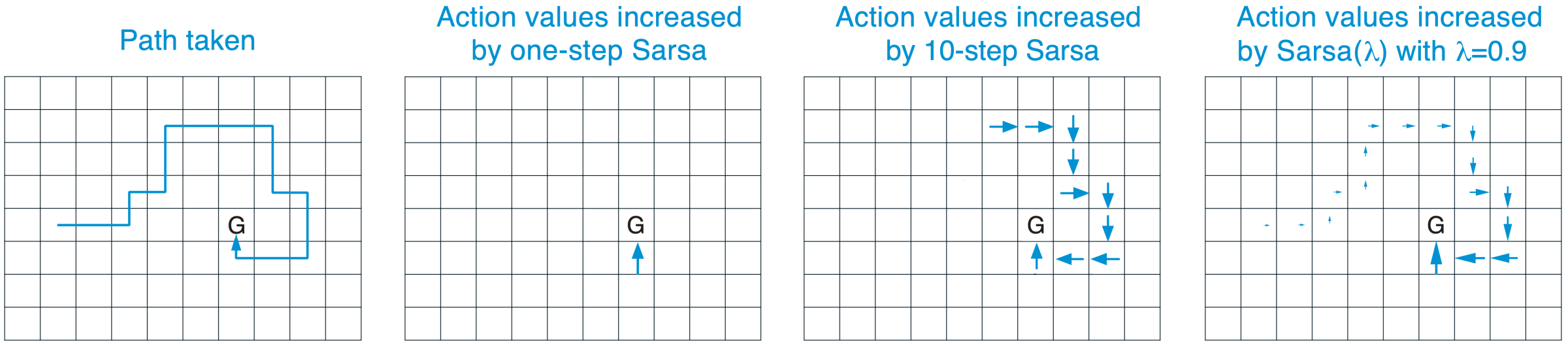
Backward view of TD(λ). Updates depend on the current TD error combined with the current eligibility traces of past events. (Sutton and Barto 1998, Figure 12.5).



Simplified representation of updating an eligibility trace of an arbitrary state in a finite MDP (Abdelwanis et al. 2026).

Example: Different SARSA variants in Gridworld

- λ can be interpreted as the discounting factor acting on the eligibility trace (see right-most panel below).
- Intuitive interpretation: more recent transitions are more certain/relevant for the current update step.



(Sutton and Barto 1998, Figure 12.1)

Summary / what you have learned

Summary / what you have learned

- TD **unites two key characteristics from DP and MC**:
  - From MC: sample-based updates (i.e., operating in unknown MDPs).
  - From DP: update estimates based on other estimates (bootstrapping).
- TD allows certain **simplifications and improvements compared to MC**:
  - Updates are available after each step and not after each episode.
  - Off-policy learning comes without importance sampling.
  - Exploits MDP formalism by maximum likelihood estimates.
  - TD prediction and control exhibit a high applicability for many problems.
- **Batch training** can be used when only limited experience is available, i.e., the available samples are re-processed again and again.
- Greedy policy improvements can lead to **maximization biases** and, therefore, slow down the learning process.
- TD **requires careful tuning of learning parameters**:
  - Step size α: how to tune convergence rate vs. uncertainty / accuracy?
  - Exploration vs. exploitation: how to visit all state-action pairs?
- n-step TD versions and TD(λ) offer a **unified view on MC and TD** learning

References

Abdelwanis, Ali, Barnabas Haucke-Korber, Darius Jakobeit, Wilhelm Kirchgässner, Marvin Meyer, Maximilian Schenke, Hendrik Vater, Oliver Wallscheid, and Daniel Weber. 2026. “Reinforcement Learning: A Comprehensive Open-Source Course.” *Journal of Open Source Education* 9 (97). The Open Journal: 306. doi:10.21105/jose.00306.

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.